学

A FULL-STACK LANGUAGE-LEARNING PLATFORM

HSK Trainer

A complete preparation platform for the Chinese HSK exams:
spaced repetition (FSRS), exam-format listening and writing practice,
grammar, classifiers, tones, and LLM-powered correction

Author Ivann Vasic

Stack FastAPI · Next.js · Postgres · Anthropic Claude

Type Technical project report

Edition Revised, November 2026

Abstract

HSK Trainer is a web application that prepares learners for the official *HSK* Chinese proficiency exams. What started as a vocabulary quiz with an LLM grader has grown into a full preparation platform organised in eight study areas: vocabulary review driven by a modern spaced-repetition scheduler (FSRS) over the complete HSK 2.0 and HSK 3.0 word lists (levels 1–5, 3,600 words); listening and writing papers that reproduce the structure of the real exams; the 210 official grammar points of levels 1–3; measure words; tones; free written expression corrected by Anthropic Claude over a live stream; a handwriting pad; and a statistics dashboard. This report describes the purpose of the system, each feature, the decoupled architecture behind it, the engineering decisions and their trade-offs, and how the whole is tested and deployed at near-zero cost. The project began as a single Streamlit prototype and was deliberately rebuilt as a typed FastAPI service with a separate Next.js front end to demonstrate production-oriented software, data and AI-integration practices.

Contents

1 Introduction and Motivation **2**

2 Product Overview **2**
2.1 Vocabulary review (Compréhension) . . . 2
2.2 Listening and writing in the exam format . . . 3
2.3 Grammar . . . 4
2.4 Classifiers . . . 4
2.5 Tones . . . 4
2.6 Free written expression . . . 4
2.7 Handwriting practice . . . 4
2.8 Dashboard and statistics . . . 4
2.9 Administration and usage limits . . . 5

3 Architecture **5**
3.1 Request lifecycle . . . 6

4 Technology Stack **6**

5 Backend Design **6**
5.1 API surface . . . 6
5.2 Data model . . . 6
5.3 Spaced repetition service . . . 7
5.4 Content pipeline: reviewed banks, validated by code . . . 8
5.5 LLM integration . . . 8

6 Frontend Design **9**
6.1 Optimistic UI: the latency win . . . 9
6.2 One exercise runner for every paper . . . 10
6.3 Speech without a speech service . . . 10
6.4 Typing Chinese . . . 10
6.5 Consuming the SSE stream . . . 10

7 Architecture Decisions and Trade-offs **11**

8 Testing and Continuous Integration **13**

9 Deployment and Operations **13**

10 Limitations and Next Steps **14**

11 Conclusion **14**

1 Introduction and Motivation

Preparing for the HSK requires several distinct skills: *recognising* vocabulary quickly, *understanding* spoken Chinese at exam speed, *producing* correct characters and sentences, and mastering the grammar points the syllabus lists for each level. Generic flashcard apps address the first poorly (fixed intervals, no production practice) and the others not at all, while past papers are scarce — especially for the new HSK 3.0 format, whose first sessions take place in 2026.

HSK Trainer was built to solve a concrete personal need — preparing the HSK 3.0 level 3 exam as a French speaker — while simultaneously serving as an engineering portfolio piece. That dual purpose shaped every decision: features were only kept if they passed the "would I use this every day?" test, and the architecture was chosen to make production practices visible (typed APIs, migrations, streaming LLM integration, CI, infrastructure-as-code).

The application interface is in French (the learner's language) and the content is Chinese. It supports **both exam versions**: HSK 2.0 (2012 lists, 2,496 words for levels 1–5) and HSK 3.0 (2025 syllabus, 3,600 words for levels 1–5). Three ideas run through the whole product:

- **Schedule what is about to be forgotten.** An FSRS scheduler selects the vocabulary to review; every other area prioritises what has never been practised, then what was failed last time.
- **Practise in the format of the exam.** Listening and writing sets follow the official papers part by part, with the same number of questions and recordings played twice.
- **Use the LLM where judgement is needed, and nowhere else.** Claude grades free production (texts, sentences), writes example sentences and extends the question bank; everything that can be checked deterministically is checked in code.

2 Product Overview

After signing in, the learner navigates between eight tabs, plus an Admin page reserved to the owner. Table 1 summarises them; the following subsections describe each one.

Tab	Skill	Content
Compréhension	Vocabulary	FSRS review of the HSK word lists, three answer modes
Écoute	Listening	Listening papers of HSK 1–3 in the real format
Écriture HSK	Writing	Writing papers of HSK 1–3, sentences graded by Claude
Grammaire	Grammar	210 official grammar points, lessons and quizzes
Classificateurs	Measure words	44 classifiers of HSK 1–3, three exercise modes
Tons	Pronunciation	Find or dictate the tone of each syllable
Expression	Free writing	Short texts corrected by Claude over a live stream
Compte	Follow-up	Dashboard, study statistics, word lists, drawings

Table 1: The study areas of the application.

2.1 Vocabulary review (Compréhension)

The core of the app. A session is configured in a few clicks and then runs entirely client-side for instant transitions.

- **Session setup** — one or several HSK levels mixed in the same session; 10, 20, 50, a custom number or the maximum available (up to 4,000 cards); and a choice of words: *all* (due cards first, with some new words), *new only*, or *revision only*.
- **Three answer modes** — *self-assessment* (the fastest, ideal on a phone); *pinyin input*, where the pinyin must be typed before judging (tone marks are optional in the comparison); and

hanzi input, the reverse direction: the French translation is shown and the word must be typed in characters. In that last mode nothing that would reveal the answer — audio, examples, drawings — appears before the answer is checked.

- **Rating** — four FSRS ratings (again / hard / good / easy) with keyboard shortcuts 1–4, progressive hints (pinyin first, then the full solution), and an optional *pinyin lock* that keeps the pinyin displayed on every card, saved on the account.
- **Study tools on each card** — a "listen" button; example sentences generated on demand by Claude (sentence, pinyin, translation, an explanation of the word and its decomposition character by character), saved per user and shown again at later reviews; and a handwriting pad.
- **HSK 2.0 / 3.0 switch** — a toggle selects the exam version. A word is a single entry listed under one level per version, so its progress is shared: switching version only changes which level the word belongs to.

2.2 Listening and writing in the exam format

The *Écoute* and *Écriture HSK* tabs reproduce the papers of levels 1–3. For HSK 3.0 the structure follows the official 2025 specification and sample papers; for HSK 2.0 it follows the 2012 sample papers. Each part keeps its order, its instruction and its number of questions.

- **Twelve question types** — picture choice, picture true/false, picture matching, word matching, short and long dialogue questions, passage questions and true/false statements for listening; missing character from pinyin, word reordering and sentence writing for writing. Pictures of the real papers are replaced by short French descriptions of the scene.
- **Recordings without audio files** — each recording is a script read by the device's own Chinese voices through the Web Speech API, one voice per speaker, played twice automatically as in the exam, with a "listen again" button.
- **A reviewed question bank** — more than 330 hand-reviewed items (about 400 questions). A build script validates every item and checks that its Chinese only uses characters from the word lists of its level.
- **"New series" by Claude** — for any part, the learner can ask for a brand-new set. Generated items go through the same parser and the same vocabulary check as the bank, and are kept for that learner.
- **Scores** — every finished set is saved, and the catalogue of parts shows the number of attempts and the best and latest scores.

Sentence writing (HSK 3, writing part 2). This part of the exam asks for a sentence built around an imposed word. The app turns it into a personalised drill with three sources: *words already seen* in the learner's own reviews, *weak words* (up to fifty words whose mastery score is below one half), and *grammar structures* — 61 major structures of levels 2–3 such as the 把 construction or 越来越. Structures never practised come first, then those failed last time. Claude grades each sentence (correctness, corrected version with pinyin and translation, a short explanation and a model sentence when wrong); for structures, the pattern is only revealed in the correction. A hint with the meaning is available on demand, and a *skip* button replaces the current item: a skipped structure is counted as mastered and goes to the back of the queue without affecting scores, while a skipped word is simply replaced.

2.3 Grammar

The 210 official grammar points of levels 1–3 (48 + 81 + 81, from the 语法等级大纲 of the 2021 standards) each have a card written for this app: a French title, an explanation, the pattern, examples, and a short quiz. The learner can practise one point or a mixed quiz, and progress is tracked per point.

2.4 Classifiers

Measure words are a classic difficulty for French speakers. The tab covers 44 nominal and verbal classifiers of HSK 1–3, each with a reference card (usage, typical nouns, examples, common traps) and 178 reviewed fill-in sentences whose distractors are unambiguously wrong. Three modes exist: choosing the classifier among options, *writing* it with no options, and a quiz on what each classifier is used for. Claude can generate ten more sentences for a classifier on request.

2.5 Tones

The *Tons* tab drills the tone of every syllable of the HSK words of the chosen levels. In the first mode the word is displayed and the learner picks the tone (1–4 or neutral) of each character; a dictation mode plays the word instead of showing its pinyin. The split of a word's pinyin into syllables is computed in the browser with `pinyin-pro`; no server call is needed beyond loading the word list.

2.6 Free written expression

The learner picks a curated subject, asks Claude for a new one, or writes their own, then submits a short text in Chinese. The correction streams back in real time: a score, the corrected version with pinyin and translation, a typed list of mistakes, strengths and a recommendation. A closing feedback loop ties this area to vocabulary: the words Claude flags as misused are automatically pushed back into the FSRS review queue. Past attempts remain browsable.

2.7 Handwriting practice

"S'entraîner à écrire" opens one 米字格 practice square per character of the word. The pad is stylus-friendly (palm rejection, pen, eraser, undo, clear). Strokes are kept as vectors normalised to their cell, so the pad can be resized without loss and the drawing is exported as a clean fixed-resolution PNG. Drawings are saved per user and per word, shown again at later reviews and gathered in a gallery on the account page.

2.8 Dashboard and statistics

The account page answers two questions: *where am I?* and *am I working regularly?*

- **Progress** — key figures (streak, success rate, words seen and mastered), progress per HSK level, a mastery-distribution chart, a forecast of upcoming reviews, and browsable lists of *mastered*, *known* and *to-review* words.
- **Activity** — a 90-day heatmap, the number of new words seen per day, and quiz and expression progression curves.
- **Study statistics** — over 7, 30 or 90 days: activity per tab, vocabulary growth, time spent, and habits by hour and weekday.

All days follow the learner's own clock: the browser sends its time zone in an `X-Timezone` header, so a review done at 07:00 in Shanghai is not attributed to the previous day by a server running in UTC.

2.9 Administration and usage limits

A read-only *Admin* page, visible only to the accounts whose e-mail is listed in an environment variable, shows site-wide figures (accounts, sign-ups per week, active users, Claude token consumption) and, for each account, activity counters and the same statistics page the user sees. It deliberately exposes counters and dates only: no password hash, no text written by users, no drawings.

Every call to Claude is metered. Each of the five kinds of call (text correction, subject generation, example generation, exercise generation, sentence grading) is limited to 15 per user per day, and to 100 for administrators. Registration can also be restricted to an allow-list of e-mails.

3 Architecture

The system is a **monorepo** with two independently deployable services that communicate over HTTPS using JSON and, for the text-correction endpoint, a Server-Sent-Events (SSE) stream.

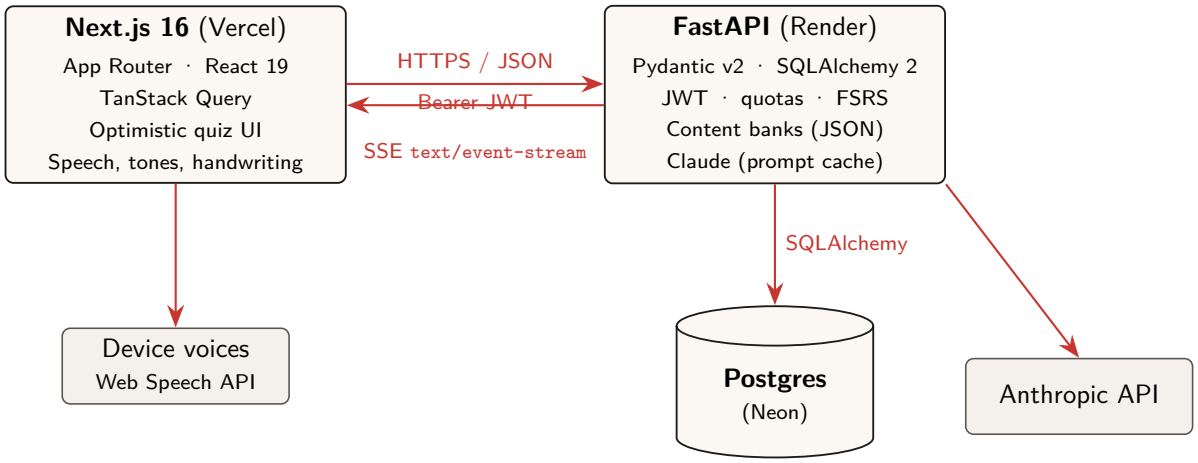


Figure 1: High-level service topology. The front end and back end deploy independently; speech synthesis happens entirely on the learner's device.

The repository layout reflects this separation:

```
backend/ FastAPI service (Python 3.12)
  app/api/v1/ auth, quizzes, srs, mastery, examples, drawings, exercises,
                classifiers, expression, account, admin
  app/services/ srs, llm, exercise, grammar, classifier, expression, stats,
                account, admin, version, drawing, example, ...
  app/models/ SQLAlchemy ORM (13 tables)
  app/schemas/ Pydantic v2 request/response models
  app/core/ settings, security (JWT, password hashing), time zones
  alembic/ 8 migrations (0001 baseline -> 0008 exercises)
  data/ word lists (HSK 2.0 and 3.0), exercise bank, grammar points,
                classifiers, structures, writing subjects
  scripts/ build + validation scripts for the content banks
  tests/ pytest, 138 tests (Anthropic mocked)
frontend/ Next.js 16 app (TypeScript)
  app/ routes: landing, signin/signup, comprehension, listening,
                writing, grammar, classifiers, tones, expression, account, admin
  components/ quiz, exercises, grammar, classifiers, tones, expression,
                writing (drawing pad), dashboard, charts, admin, ui
  lib/ typed API client, auth context, SSE consumer, speech,
                tones, handwriting, pinyin helpers
  tests/ Vitest + Testing Library, 87 tests
```

3.1 Request lifecycle

Every authenticated request carries a JSON Web Token in an `Authorization: Bearer` header. A reusable FastAPI dependency decodes and validates it, resolves the user, and — on the endpoints that call Claude — enforces the per-user daily limit before the handler runs. Admin endpoints add a second dependency that returns 403 to everybody else.

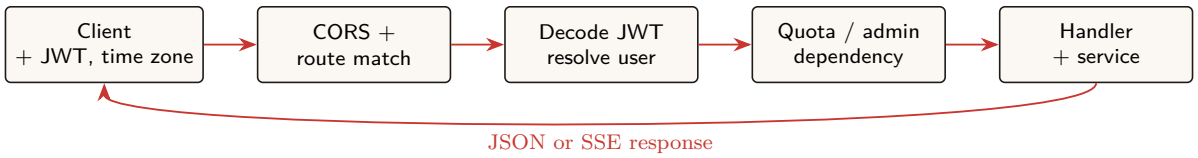


Figure 2: Request pipeline. Auth, quotas and admin rights are dependencies, so handlers stay free of cross-cutting concerns.

4 Technology Stack

Layer	Choice	Rationale
API	FastAPI + Pydantic v2	Typed I/O, auto OpenAPI, SSE streaming
ORM	SQLAlchemy 2.x	Mature, explicit, Postgres + SQLite
Migrations	Alembic	Versioned schema, idempotent migrations run at deploy
Auth	JWT (HS256) + PBKDF2	Stateless, no session store
Scheduler	FSRS (py-fsrs 6)	ML-based, outperforms SM-2
LLM	Claude Haiku 4.5	Cheap, fast, prompt-cache friendly
Content	YAML sources → JSON	Reviewed by hand, validated by build scripts
Front end	Next.js 16 / React 19	App Router, mature ecosystem
Server state	TanStack Query	Caching, retries, optimistic updates
Styling	Tailwind v4	CSS-first tokens, light and dark themes
Forms	react-hook-form + zod	Typed validation
Charts	Recharts	Composable charts for the dashboard
Animation	Framer Motion	Card transitions
Speech	Web Speech API	Free, on-device Chinese voices, works offline
Pinyin	pinyin-pro	Syllable splitting for the tone drills
Database	Postgres (Neon)	Free tier, autosuspend, branching
Tests	pytest, Vitest	Offline suites, Testing Library for components
CI	GitHub Actions	pytest + Vitest + production build

Table 2: Stack and the reasoning behind each layer.

5 Backend Design

5.1 API surface

The API is versioned under `/v1` and fully described by an auto-generated OpenAPI document (Swagger UI at `/docs`).
Two unauthenticated probes complete it: `/healthz`, a liveness check that does not touch the database, and `/healthz/db` for manual checks of the connection.

5.2 Data model

Thirteen tables, grouped by concern:

- **Accounts** — `users`, `user_settings` (key/value: HSK version, pinyin lock).

Group	Endpoints
`auth`	register, login, me (with the computed admin flag)
`quizzes`	catalogue of levels per HSK version, entries of a level
`srs`	rate a card, fetch a session (due / new / revision), counters
`mastery`	per-word state for the word lists
`examples`	list and generate example sentences for a word
`drawings`	save, list, delete handwriting drawings; gallery
`exercises`	catalogue of parts, practice set, results, generation by Claude, sentence and structure grading, skip, grammar points
`classifiers`	reference cards, quiz (three modes), sentence generation
`expression`	subjects, subject generation, streamed correction, attempts, quota
`account`	dashboard overview, study statistics, settings (version, pinyin lock)
`admin`	accounts overview and per-account statistics (read-only)

Table 3: Endpoint groups of the `/v1` API.

- **Vocabulary** — `entries` (one row per word), `quizzes` (one per level and version) and `quiz_entries`, the many-to-many link that lets a word belong to a different level in each version.
- **Spaced repetition** — `user_vocab_mastery` (the FSRS card state per user and word) and `review_log` (one row per answer: rating, mode, time spent, whether the card was new or due).
- **Study material** — `word_examples`, `word_drawings`.
- **Exercises** — `exercise_results` (one row per finished set, with the detail of each answer) and `generated_exercises` (items written by Claude for a learner).
- **Expression and metering** — `expression_attempts`, `user_daily_usage` (calls and tokens per user, day and kind).

The schema evolves through eight Alembic migrations, applied automatically at each deployment before the server starts; if one fails, the new instance never starts and the previous deployment stays live. Two of them are data migrations worth mentioning: the switch of the vocabulary to the HSK 3.0 syllabus *without losing any progress*, and the introduction of HSK 2.0 next to it by re-modelling "a word belongs to a level" as a link table. Because the project replaced an earlier app that already owned a populated production database, the baseline was *stamped* rather than replayed, and later migrations are idempotent. Tests run against a throwaway SQLite database.

5.3 Spaced repetition service

The scheduler is isolated behind a service module. The public API speaks the standard SRS rating vocabulary (`again`/`hard`/`good`/`easy`); internally that maps to FSRS `Rating` values, and the FSRS card state is serialised onto the mastery row so scheduling survives across sessions. Every answer is also appended to `review_log`, the single source for streaks, success rates, the heatmap and the study statistics.

Listing 1: FSRS rating, simplified — the scheduler owns interval/stability, the row is just persistence.

```
def rate_entry(db, *, user_id, entry_id, rating: RatingEnum, mode=None,
               elapsed_ms=None) -> MasterySnapshot:
    record = _get_or_create_mastery(db, user_id, entry_id)
    card = _build_card_from_record(record) # rebuild FSRS state
    new_card, _ = _SCHEDULER.review_card(
        card, _RATING_TO_FSRS[rating], review_datetime=datetime.now(timezone.utc)
    )
    _write_card_into_record(record, new_card, rating) # stability, due, ...
```

```
    db.add(ReviewLog(user_id=user_id, entry_id=entry_id, mode=mode,
                     rating=_RATING_TO_FSRS[rating].value, elapsed_ms=elapsed_ms))
    return MasterySnapshot(entry_id=record.entry_id,
                           next_review_at=record.fsrs_due_at,
                           stability_days=record.fsrs_stability)
```

5.4 Content pipeline: reviewed banks, validated by code

Exam questions, grammar cards, classifier cards and grammar structures are authored as YAML files, one per level, and compiled to JSON by build scripts. The scripts are the quality gate: they parse every item with the same schema the API uses, shuffle the options (the right answer is written first in the sources), and — the key check — verify that the Chinese of each item only uses characters from the word lists of its level, recording which exam versions the item fits. A question that uses a word above the learner's level cannot reach production.

Listing 2: Structure priority, simplified — never practised first, then failed last time, then the rest.

```
def structure_items(db, user_id, count, rng):
    history = structure_history(db, user_id) # last result per structure
    never = [s for s in structures() if s["id"] not in history]
    failed = [s for s in structures() if history.get(s["id"]) is False]
    passed = [s for s in structures() if history.get(s["id"]) is True]
    for group in (never, failed, passed):
        rng.shuffle(group)
    return (never + failed + passed)[:count]
```

5.5 LLM integration

Claude is used for six tasks, all through one service module: correcting a text, proposing a writing subject, writing an example sentence for a word, generating a new series of exam questions, generating classifier sentences, and grading a sentence written with an imposed word or structure. Four properties make the integration production-grade.

Structured output. Every prompt asks for JSON with a fixed shape; the response is parsed and validated before anything is stored or shown. Generated exam items go through the same parser and vocabulary check as the hand-written bank, so the model cannot introduce a malformed question or an out-of-level word.

Prompt caching. The static part of each prompt is wrapped in an ephemeral cache block. For exercise generation, the cached block is the whole authorised vocabulary of the level (up to 1,000 words), so repeated generations pay the cheap cache-read price instead of re-billing the list every call.

Listing 3: Prompt caching — the vocabulary of the level is cached; only the request varies.

```
message = client.messages.create(
    model=get_model_id(), max_tokens=6000,
    system=[{"type": "text", "text": EXERCISE_SYSTEM_PROMPT},
           {"type": "text", "text": vocab_block,
            "cache_control": {"type": "ephemeral"}}],
    messages=[{"role": "user", "content": user_msg}],
)
```

Streaming. The text-correction endpoint streams the model output to the browser as typed SSE events. The generator opens its own database session so it can persist the attempt and

re-queue misused vocabulary *after* the stream completes, without holding a request-scoped transaction open for the whole response.

Listing 4: SSE generator, simplified — events: `start`, `partial*`, `complete`, `persisted` (or `error`).

```python
def _stream_correction_response(*, user_id, limit, level, subject, user_text):
    db = SessionLocal()
    try:
        usage = get_daily_usage(db, user_id, "expression_correction")
        if usage.count >= limit: # race-safe re-check
            yield _sse("error", {"message": "Quota quotidien atteint."})
            return
        for event in llm_service.stream_correction(user_text, level=level,
                                                   subject=subject):
            yield _sse(event.type, event.data) # start / partial / complete
        attempt = expression_service.record_attempt(db, ...)
        relearned = queue_entries_for_relearn(db, user_id, misused_entry_ids)
        db.commit()
        yield _sse("persisted",
                   {"attempt_id": attempt.id, "relearned_count": relearned})
    finally:
        db.close()
```

Metering. Each call increments a per-user daily counter that also records input and output tokens. The limit is resolved per user, which is how administrators get a higher allowance without a separate code path.

Listing 5: Daily quotas as a dependency factory.

```python
DAILY_QUOTA, ADMIN_DAILY_QUOTA = 15, 100

def daily_quota(user: User, kind: str) -> int:
    if kind not in QUOTA_KINDS:
        return 0 # unknown kind: unlimited
    return ADMIN_DAILY_QUOTA if is_admin(user) else DAILY_QUOTA

def enforce_quota(kind: str):
    def _check(user=Depends(get_current_user), db=Depends(get_db)) -> User:
        limit = daily_quota(user, kind)
        usage = get_daily_usage(db, user.id, kind)
        if limit and usage.count >= limit:
            raise HTTPException(429, f"Quota quotidien atteint ({usage.count}/{limit}).")
        return user
    return _check
```

6 Frontend Design

The front end is a Next.js App Router application written in TypeScript. Auth state lives in a React context backed by `localStorage`; server state (quizzes, due cards, mastery, exercise sets, statistics) is cached by TanStack Query. A small typed `fetch` wrapper attaches the Bearer token and the time zone, and mirrors the backend's Pydantic schemas. The interface is mobile-first, supports light and dark themes, and installs to a phone home screen as a PWA.

6.1 Optimistic UI: the latency win

The single most impactful UX decision. In the original Streamlit prototype, every interaction triggered a full server rerun, so the gap between answering a card and seeing the next one was 2–5 seconds. The rewritten quiz prefetches the whole session, advances the local index *immediately* on a rating, and fires the persistence request in the background with retry.

Listing 6: Optimistic advance — the next card renders before the network call resolves.

```
const recordRating = useCallback((rating: RatingEnum, meta?) => {
  setState(prev => ({
    ...prev,
    currentIndex: prev.currentIndex + 1, // advance now
    history: [...prev.history, { entry: prev.questions[prev.currentIndex],
                                 rating, ...meta }],
  }));
}, []);
// the page then fires postRate() via a background mutation (retry x2)
```

The perceived transition dropped from seconds to well under 50 ms, with persistence failures surfaced non-blockingly through a toast.

6.2 One exercise runner for every paper

Listening, writing, grammar and classifier sets all run through a single runner component. It receives a set of typed items, renders the right question component for each type, keeps the answers, shows the correction after each question and saves the result at the end. Adding a question type means adding one parser entry on the server and one component on the client. The runner also owns the *skip* behaviour of the sentence-writing drill: the skipped item is replaced in place, and the list of items already drawn or skipped is sent to the server so the replacement is never a repeat.

6.3 Speech without a speech service

Audio is produced by the browser's own text-to-speech. A small module picks the best Mandarin voice available (mainland Mandarin first, local voices before network ones — local voices work offline and do not depend on services blocked in mainland China) and a script player reads a dialogue line by line, assigning a different voice to each speaker when the device has several.

6.4 Typing Chinese

Accepting typed Chinese has two pitfalls, both handled in the hanzi-input mode. With a Chinese input method, the Enter key first confirms the candidate characters, so a naive "Enter submits" handler would check a half-typed answer; the handler ignores Enter while a composition is in progress. And input methods leave spaces or punctuation behind, so the comparison ignores them.

Listing 7: Comparing a typed word with the expected characters.

```
export function hanziMatches(userInput: string, expected: string): boolean {
  const clean = (s: string) =>
    s.normalize("NFKC").replace(/[\s\p{P}\p{S}]/gu, "");
  const left = clean(userInput);
  return left.length > 0 && left === clean(expected);
}
// in the input: if (e.key === "Enter" && !e.nativeEvent.isComposing) check();
```

6.5 Consuming the SSE stream

The browser's native `EventSource` cannot send an `Authorization` header, so the front end reads the stream manually with `fetch` and a `ReadableStream` parser.

Listing 8: Custom SSE reader — handles auth headers, which native EventSource cannot.

```
const reader = response.body.getReader();
const decoder = new TextDecoder();
```

```
let buffer = "";
while (true) {
  const { done, value } = await reader.read();
  if (done) break;
  buffer += decoder.decode(value, { stream: true });
  let i; while ((i = buffer.indexOf("\n\n")) !== -1) {
    const block = buffer.slice(0, i); buffer = buffer.slice(i + 2);
    handlers.onEvent(parseSseBlock(block)); // {event, data}
  }
}
```

While Claude streams, the UI shows an "analysing" indicator with a live token counter; on the `complete` event it swaps to the fully structured correction with an animated reveal. The visual identity (a cinnabar/ink/paper palette inspired by Chinese seal art, with 学 as the mark) carries through to the PWA manifest.

7 Architecture Decisions and Trade-offs

Every significant choice was made explicitly. The most consequential are recorded below with the alternative that was rejected and why.

FSRS over SM-2 or a hand-rolled heuristic

The original prototype used a fixed-ratio bucket heuristic (45 % review / 50 % new). FSRS models per-card stability and difficulty and schedules each review just before predicted forgetting, which is the entire point of spaced repetition. SM-2 (classic Anki) was the simpler alternative but is empirically weaker; a bespoke algorithm would have been unjustifiable reinvention. FSRS is isolated behind a service boundary so it can be tuned or swapped without touching the API.

Optimistic UI over server-driven rendering

Keeping the quiz server-authoritative (the Streamlit model) was simplest but produced the 2–5 s stall that motivated the rewrite. Prefetching the session and advancing client-side trades a small amount of state-reconciliation complexity for an order-of-magnitude better perceived latency — the correct trade for a drilling interaction repeated hundreds of times per session.

One word, one entry: levels as a link table

Supporting HSK 2.0 and 3.0 could have meant two separate vocabularies. Instead a word is a single entry linked to one level per version. Progress is therefore shared: a learner who switches version, or whose exam changes format, keeps every review. The cost was a data migration of a live database, written to be idempotent and covered by a dedicated test.

A reviewed question bank first, generation second

Generating every exercise with the LLM would have been faster to build, but a wrong answer key in an exam drill is worse than no drill. The bank is written and reviewed by hand, and Claude only *extends* it on request; in both cases the same code validates the shape of the item and the level of its vocabulary. The model is trusted for fluency, never for format or scope.

Browser speech synthesis over recorded audio or a TTS API

Recorded audio would mean hundreds of files and none for generated questions; a cloud text-to-speech API would add a cost, a key and a network dependency. The Web Speech API is free, works offline, reads any generated script and is available on every phone. The trade-off is accepted and documented: voice quality depends on the device, and some desktops need a Chinese language pack.

An append-only review log

The first dashboard was computed from the latest state of each word, which made streaks and success rates approximate. Recording every answer in `review_log` costs one small insert per card and makes every statistic exact and every future chart possible. For the days before the log existed, the heatmap falls back to the old approximation so earlier history does not vanish.

Keep a Python backend; do not rewrite in TypeScript

A single Next.js full-stack app (API routes) would have removed a service and a deployment. It was rejected to preserve the Python data/AI stack (SQLAlchemy, Alembic, the FSRS library, the Anthropic SDK) — the part of the project most relevant to its goals — and to keep a clean, independently testable API boundary.

Render free tier + keep-warm cron, with a database-free probe

The FastAPI app runs on Render's free tier and a 10-minute cron ping to `/healthz` prevents the idle spin-down. That probe originally ran a `SELECT 1`; since Neon suspends its compute after five idle minutes, the ping kept the database awake around the clock and consumed almost the whole monthly free allowance. The probe no longer touches the database (a separate `/healthz/db` exists for manual checks), and Neon wakes in under a second on the next real request. A reminder that a "harmless" health check is a workload.

JWT in localStorage with a Bearer header

An `httpOnly` cookie is the stricter choice against XSS but adds cross-site cookie and CSRF complexity between a Vercel front end and a Render API. For a small study tool the localStorage + Bearer approach is a deliberate, scoped simplification, isolated in one auth module so it can be hardened later without touching call sites.

Administrators by environment variable, not by database role

Admin rights are computed on every request from an allow-list of e-mails in the server configuration; nothing is stored and nothing can be granted through the API. For a site with a single owner this removes a whole class of privilege bugs, and the Admin page is read-only by design.

Typed SSE events over raw text or no streaming

A non-streamed response was simplest but loses the real-time feedback that makes a multi-second LLM call feel responsive. Streaming raw text would have pushed JSON parsing

into the UI. Typed events (`start` / `partial` / `complete` / `persisted`) keep the transport self-describing and let the client switch cleanly from a progress state to the structured result.

8 Testing and Continuous Integration

The backend has a pytest suite of **138 tests** that runs entirely offline: the Anthropic client is replaced by a fake that mimics both the one-shot `messages.create` call and the streaming context manager. It covers authentication, session selection, the review log, the HSK 3.0 and version migrations, the exercise catalogue and scoring, generation and its validation, sentence and structure grading, the skip behaviour, classifiers, drawings, quotas, statistics and the admin restrictions — without a network or a paid API call.

The front end has **87 Vitest tests**: unit tests for the pure logic (the tone-insensitive pinyin comparator, tone splitting, voice selection, stroke handling) and component tests with Testing Library for the quiz setup and its three modes, the exercise runner, the classifier and tone sessions, the dashboard and the admin page. New features are additionally checked in a real browser at desktop and phone sizes before release.

GitHub Actions runs both suites plus a production Next.js build on every push and pull request, so a type error or a broken test is caught before it can be deployed. A subtle bug surfaced and fixed this way — a SQLite write lock when the SSE generator opened a second session while a request-scoped transaction was still open — is exactly the class of issue an offline integration test is meant to catch.

9 Deployment and Operations

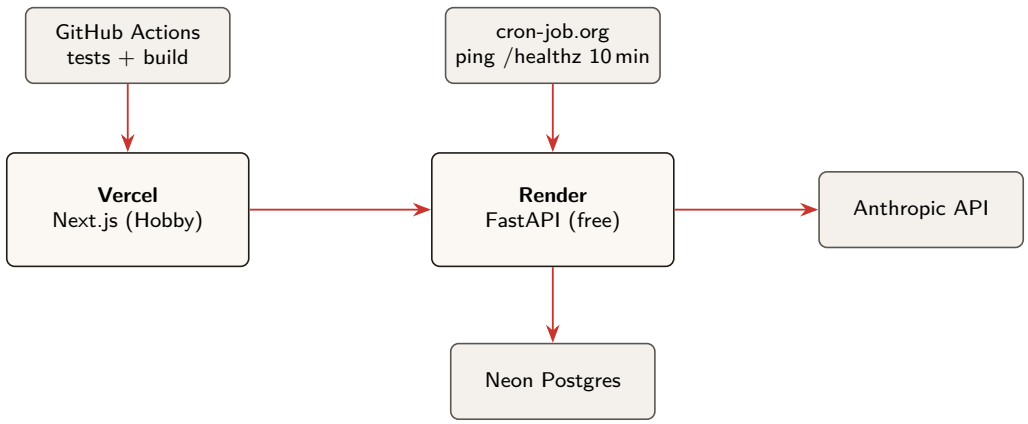


Figure 3: Deployment topology. Every component runs on a free tier; the cron ping keeps the free Render instance from sleeping.

The entire production stack runs at a recurring cost of approximately **€0**. The only variable cost is Anthropic usage, bounded by the daily quotas: with prompt caching, a written-expression correction costs on the order of $0.003. Infrastructure is declared as code in `render.yaml`: the build command, the start command (which applies pending migrations before serving), the health check and the list of environment variables, with secrets kept in the hosting dashboard. The front end redeploys automatically on every push to `main`; the back end is redeployed from the Render dashboard.

Operational safeguards, in short: migrations that fail keep the previous version live; registration can be closed to an allow-list; every Claude call is metered per user; uploads are bounded (drawings are validated PNGs of at most 1 MB, 2,000 per user); and text inputs have length

limits.

10 Limitations and Next Steps

- **Exam coverage** — listening and writing papers cover levels 1–3; the reading section and levels 4–5 are not reproduced yet. One part of the HSK 3.0 level 2 writing paper is listed but unavailable, because its official instruction is not published in text.
- **Pictures** — the pictures of the real papers are replaced by written descriptions, which is slightly easier than the exam.
- **Speech** — voice quality depends on the device; there is no speaking practice or pronunciation scoring.
- **Quota reset** — daily limits reset at midnight UTC rather than at the user's local midnight.
- **Answer matching** — the hanzi-input mode accepts only the expected word, not a synonym with the same translation; the learner rates the card themselves, which absorbs the ambiguity.

11 Conclusion

HSK Trainer has grown from a vocabulary quiz with an LLM grader into a complete preparation platform covering vocabulary, listening, writing, grammar, classifiers, tones and free expression for both versions of the exam. It also demonstrates a complete path from a quick prototype to a production-shaped application: a typed, versioned API; a data layer evolved through tested migrations without losing user progress; reviewed content validated by code; an LLM integration with structured output, caching, streaming and cost controls; an optimistic, mobile-first front end; offline test suites with CI; and zero-cost infrastructure-as-code. The guiding constraint — that it had to be a tool worth using daily *and* a credible engineering artefact — pushed the design toward explicit, defensible trade-offs rather than defaults. The result is an application whose every layer can be explained and justified.

Ivann Vasic – HSK Trainer – FastAPI · Next.js · Postgres · Anthropic Claude